Prometheus — ESP32 Wi-Fi / OLED

Configuração pela web

1. Acesse `http://IP_DO_ESP32/prometheus` ou clique em **Prometheus** no portal. O painel e a configuração ficam sempre na **porta 80**.
2. Informe o endereço do servidor Prometheus: IPv4 ou DNS, sem `http://`, caminho ou porta. Exemplo: `192.168.1.10` ou `prometheus.local`.
3. Escolha a **porta de métricas do ESP32**: 1..65535, padrão **9120**. Essa não é a porta de acesso ao servidor Prometheus.
4. Escolha o intervalo de coleta: 5..3600 segundos, padrão **15**.
5. Salve. A porta é aplicada cerca de um segundo depois, sem reboot e sem redirecionar o navegador. As preferências são restauradas após reiniciar.
6. Copie o YAML exibido ou baixe `/prometheus.yml`, adicione o job à configuração do servidor Prometheus e recarregue essa configuração.

O modelo é **pull**: o servidor Prometheus acessa `http://IP_DO_ESP32:PORTA/metrics`. O ESP32 não envia dados ativamente e não usa Pushgateway. O endereço informado é uma referência persistente; salvar no ESP32 não altera a configuração do servidor. O intervalo só passa a valer na coleta quando o YAML é aplicado ao Prometheus.

Separação das portas

Serviço	Porta
Painel, redes, terminal, formulário Prometheus e download YAML	**80**, fixa
Endpoint `/metrics`	Configurada nas preferências, padrão **9120**

Se a porta de métricas for 80, o endpoint compartilha o servidor do painel. Nas demais portas, o servidor de coleta atende apenas `/metrics`; essa rota retorna 404 na porta 80. As mesmas regras valem no AP **ESP32_CONFIG**, IP **192.168.4.1**, e após reiniciar.

Reserve um IP para o ESP32 no DHCP. O YAML usa o IP da estação quando conectada e o IP do AP quando offline; gere novamente o YAML se o endereço do alvo mudar. Não há autenticação nos endpoints: use somente uma rede confiável, sem expor o dispositivo diretamente à Internet.

Exemplo de configuração no servidor

Substitua o endereço pelo IP real do ESP32 e a porta pelo valor salvo:

```
scrape_configs:
  - job_name: 'esp32_wifi'
    scrape_interval: 15s
    metrics_path: /metrics
```

```
  static_configs:
    - targets: ['192.168.1.42:9120']
```

Se já houver `scrape_configs:`, adicione apenas o item `- job_name`, sem duplicar a seção. Use um nome de job único ou reúna os dispositivos nos targets existentes.

Persistência e erros

O namespace Preferences/NVS **`telemetry`** contém:

- `settings`: endereço do servidor, porta de métricas e intervalo, em um registro.
- `boots`: contadores de inicializações e resets por watchdog.

Uma falha ao salvar retorna HTTP **503** e preserva a configuração em RAM. Entrada inválida retorna **400**. Configurações ausentes usam os padrões; configurações inválidas ou NVS indisponível são informadas na Serial. Os contadores persistentes são gravados uma vez por boot, não em cada coleta. Se não puderem ser carregados/gravados, suas séries são omitidas e há aviso na Serial e na página de configuração.

`reset_wifi` e a limpeza por pressão longa do botão apagam o namespace `wifi`, mas **não apagam** as preferências Prometheus nem os contadores de `telemetry`. Uma limpeza completa da NVS por ferramentas externas pode removê-los.

Métricas publicadas

Todas as séries têm o label `device`, com o MAC da interface STA, estável também no AP. Identificações são gauges com valor 1 e informação nos labels. O formato de exposição é Prometheus **0.0.4**.

Grupo	Métrica	Significado
Identificação	`esp_info`	Labels `chip` e `model`.
Identificação	`esp_firmware_version`	Labels `version` e `build` (data/hora da compilação).
Identificação	`esp_ip_address`	Labels `ip` e `interface`; uma série por interface ativa STA/AP.
Identificação	`esp_mac_address`	MAC STA no label `mac`.
Disponibilidade	`esp_status_online`	1 com Wi-Fi STA conectado; 0 sem conexão. Não testa Internet.
Disponibilidade	`esp_uptime_seconds`	Tempo desde boot, em segundos, com timer de 64 bits.
Disponibilidade	`esp_last_restart_reason`	Código numérico de `esp_reset_reason_t` no label `reason`.
Disponibilidade	`esp_boot_count`	Inicializações acumuladas na NVS.
Wi-Fi	`esp_wifi_rssi_dbm`	RSSI em dBm; omitida sem STA conectado.

Grupo	Métrica	Significado
Wi-Fi	`esp_wifi_channel`	Canal da conexão; omitida sem STA conectado.
Wi-Fi	`esp_wifi_disconnect_count`	Eventos de desconexão desde boot, incluindo tentativas e desconexões deliberadas.
Wi-Fi	`esp_wifi_tx_power`	Limite de potência de transmissão configurado no driver, convertido para dBm; não mede consumo. Omitida se a leitura falhar ou sem STA conectado.
Processador	`esp_cpu_frequency_mhz`	Frequência atual em MHz.
Processador	`esp_cpu_usage_percent`	Uso agregado dos núcleos, 0..100%; ver limitações abaixo.
Processador	`esp_task_count`	Número atual de tarefas FreeRTOS.
Memória	`esp_heap_free_bytes`	Heap livre.
Memória	`esp_heap_total_bytes`	Heap total.
Memória	`esp_heap_minimum_free_bytes`	Menor heap livre desde boot.
Memória	`esp_psram_free_bytes`	PSRAM livre; zero quando não disponível.
Memória	`esp_psram_total_bytes`	PSRAM total; zero quando não disponível.
GPIO	`gpio_digital_input`	Nível lógico do botão GPIO0; label `pin`: 0 pressionado, 1 solto.
GPIO	`gpio_digital_output`	Nível lógico instantâneo do LED GPIO2; label `pin`.
Sistema	`esp_reboot_count`	Inicializações acumuladas menos a primeira, incluindo ciclos de energia.
Sistema	`esp_watchdog_resets`	Resets acumulados pelos watchdogs de tarefa, interrupção e geral.
Sistema	`esp_free_rtos_tasks`	Mesmo número de tarefas de `esp_task_count`.
Sistema	`esp_flash_usage_percent`	Tamanho do firmware dividido pela capacidade da partição em execução; não é uso do chip flash inteiro.
Aplicação	`application_requests_total`	Counter de requisições HTTP concluídas em ambos os servidores.

Grupo	Métrica	Significado
Aplicação	`application_errors_total`	Counter de requisições HTTP com status 4xx/5xx ou sem código de resposta.
Aplicação	`application_response_time_ms`	Gauge de duração do tratamento da última requisição concluída.
HTTP	`esp_http_rejected_requests_total`	Clientes sem iniciar pedido no prazo ou pedidos rejeitados pelo parser; label `server="portal"` ou `server="metrics"`.
HTTP	`esp_http_aborted_responses_total`	Respostas normais interrompidas por prazo de envio ou erro TCP; mesmos labels.
Adicional	`esp_cpu_temperature_celsius`	Temperatura interna da CPU quando válida; não é temperatura ambiente.
OLED	`esp_oled_events_dropped_total`	Eventos de tela descartados por fila cheia, acumulados na RAM desde o boot.
UDP	`esp_udp_listener_up`	1 após abertura local bem-sucedida da porta 4210; 0 em falha/recuperação. Não comprova entrega pela rede.
UDP	`esp_udp_packets_received_total`	Datagramas recebidos, incluindo comandos rejeitados por tamanho.
UDP	`esp_udp_replies_sent_total`	Respostas aceitas para envio pela pilha UDP; não confirma recebimento pelo cliente.
UDP	`esp_udp_receive_errors_total`	Erros detectados no socket de leitura, leituras incompletas ou datagramas acima do limite.
UDP	`esp_udp_send_errors_total`	Falhas ao preparar/enviar uma resposta UDP.
UDP	`esp_udp_bind_errors_total`	Tentativas malsucedidas de abrir a porta UDP.
UDP	`esp_udp_rebinds_total`	Reaberturas bem-sucedidas após a primeira abertura.
UDP	`esp_udp_last_receive_uptime_seconds`	Uptime no recebimento do último datagrama; omitida antes do primeiro pacote.

Grupo	Métrica	Significado
Clima Porto Alegre	`sensor_temperature_celsius`	Temperatura do ar a 2 m do Open-Meteo, em °C; labels `source="open_meteo"` e `location="porto_alegre"`.
Clima Porto Alegre	`sensor_humidity_percent`	Umidade relativa do ar a 2 m do Open-Meteo, em %; mesmos labels.
Clima Porto Alegre	`esp_weather_up`	1 se a última consulta foi válida, o cache está recente e o Wi-Fi conectado; 0 caso contrário.
Clima Porto Alegre	`esp_weather_data_age_seconds`	Idade do dado desde o horário `current.time` da API; omitida antes do primeiro dado válido.
Clima Porto Alegre	`esp_weather_data_timestamp_seconds`	Horário Unix do dado climático fornecido pela API.

Clima de Porto Alegre e consultas do Grafana

O firmware consulta o Open-Meteo para as coordenadas de Porto Alegre (-30.0346, -51.2177), com `current=temperature_2m,relative_humidity_2m`. São dados de modelo meteorológico, não leituras de um sensor físico conectado ao ESP32. A temperatura interna da CPU continua separada em `esp_cpu_temperature_celsius`.

A consulta HTTPS é executada por uma tarefa própria a cada dez minutos, com nova tentativa após um minuto em falha. Requer Internet, DNS, relógio sincronizado e o bundle de certificados do Arduino-ESP32 3.3.12. Não é feita durante o scrape: `/metrics` lê um cache sincronizado, sem esperar pelo servidor meteorológico. O parser cJSON é o componente já incluído no ESP-IDF; não requer ArduinoJson.

Antes da primeira resposta válida, temperatura, umidade e idade são omitidas, e `esp_weather_up` vale 0. Em falha posterior, o último dado é preservado e sua idade continua aumentando. Temperatura e umidade são omitidas quando o dado passa de uma hora de idade; a idade e o timestamp continuam visíveis. O cache não é gravado na NVS e reinicia com a placa. Nenhum zero climático é publicado para substituir falhas. Um valor real de 0 °C ou 0% continua válido.

As consultas solicitadas podem ser usadas diretamente no Grafana:

```
sensor_temperature_celsius{source="open_meteo",location="porto_alegre"}
and on(instance, job) esp_info{model="$device"}
```

```
sensor_humidity_percent{location="porto_alegre"}
and on(instance, job) esp_info{model="$device"}
```

Para o painel de idade da imagem:

```
esp_weather_data_age_seconds{source="open_meteo",location="porto_alegre"}
and on(instance, job) esp_info{model="$device"}
```

`$device` deve conter o modelo `ESP32-WiFi-OLED`, conforme o label `model` de `esp_info`. `instance` e `job` são adicionados pelo Prometheus durante a coleta; não devem ser fixados no firmware. Todas essas séries saem do mesmo endpoint que `esp_info` e também têm o label `device` com o MAC da placa. Assim, a consulta antiga `esp_weather_data_age_seconds and on(device) esp_info{model="$device"}` também funciona. O firmware não modifica automaticamente o dashboard do Grafana.

Limitações e interpretação

- CPU é amostrada a cada cinco segundos e requer duas amostras válidas. É omitida durante aquecimento, sem estatísticas de runtime do SDK, quando faltam tarefas idle ou quando a amostra é inválida. O cálculo soma os tempos idle dos núcleos e normaliza pela capacidade total; a escala continua 0..100%, não 0..200%.
- Contadores Wi-Fi e HTTP reiniciam no boot. Os de boot/watchdog persistem até limpeza da NVS e saturam no limite de 32 bits.
- O scrape atual entra nos contadores HTTP somente após terminar sua resposta. A duração inclui tratamento e envio, não leitura inicial da requisição nem latência completa do cliente; não é média ou percentil.
- O nível do GPIO é uma leitura instantânea; não contabiliza pulsos entre coletas. I2C e ADC não são reconfigurados para gerar telemetria.
- PSRAM igual a zero significa ausência de PSRAM disponível, não erro fabricado.
- Um dispositivo desligado não publica seu estado. Use `up` do Prometheus para disponibilidade; `esp_status_online` informa apenas a conexão STA.

Métricas solicitadas ainda indisponíveis

Conforme o escopo escolhido, estas séries **não são publicadas** até existir hardware/configuração e integração reais. Não são preenchidas com zeros ou valores simulados.

Grupo	Métricas	Integração necessária
Energia	`esp_voltage_v`, `esp_current_ma`, `esp_power_w`	Medidor elétrico e leitura calibrada.
Bateria	`esp_battery_percent`, `esp_battery_voltage_v`	Bateria monitorada e circuito/calibração.
Ambiente local	`sensor_pressure_hpa`, `sensor_air_quality` e leituras locais de temperatura/umidade	Sensores ambientais, pinos e drivers. Temperatura/umidade de Porto Alegre vêm do Open-Meteo, conforme acima.
Relés	`relay_state`, `relay_activation_count`, `relay_runtime_seconds`	Relés definidos e instrumentação dos acionamentos/tempo ligado.

Grupo	Métricas	Integração necessária
ADC	`gpio_analog_value`	Entrada analógica definida, circuito e calibração.
MQTT	`mqtt_connected`, `mqtt_messages_sent`, `mqtt_messages_received`, `mqtt_publish_errors`	Cliente MQTT, broker e contagem dos eventos reais.

A temperatura interna do chip não é publicada como `sensor_temperature_celsius`. O projeto consulta o Open-Meteo para Porto Alegre; não usa MQTT.

Diagnóstico

Watchdog com rastro de atendimento HTTP

O portal e o servidor de métricas usam `BoundedWebServer`, uma adaptação local que mantém o parser, as rotas e os middlewares do WebServer do core instalado. Clientes sem dados iniciais são encerrados após 1,5 segundo sem bloquear o ciclo enquanto aguardam. A inatividade de leitura é configurada em 250 ms, em vez de cinco segundos. Cabeçalhos e formulários fragmentados continuam sendo lidos pelo parser original; os limites internos de fase/corpo desse parser continuam valendo. Esses prazos não constituem um limite absoluto para todo o tratamento HTTP.

As respostas normais dos handlers String usam envio TCP não bloqueante em blocos de até 1024 bytes, com pausa entre tentativas e prazo compartilhado de um segundo para cabeçalho e corpo. Se o cliente não acompanha, a conexão é encerrada e o próximo scrape pode tentar novamente. Essa adaptação é para as rotas atuais; não oferece streaming, SSE ou garantia de prazo para handlers adicionais ou para respostas de erro enviadas internamente pelo parser do core.

`esp_http_rejected_requests_total` e `esp_http_aborted_responses_total` permitem acompanhar essas interrupções, separadas por `server="portal"` e `server="metrics"`. Os valores ficam na RAM e são amostrados quando `/metrics` monta sua resposta; uma falha durante o envio do próprio scrape aparece na próxima coleta válida.

O rastro RTC diferencia agora portal HTTP, servidor HTTP de métricas, montagem das métricas, envio HTTP das métricas e manutenção de Wi-Fi/NVS. A última etapa continua sendo uma pista: não substitui o backtrace nem identifica sozinha qual tarefa provocou o watchdog.

Prometheus funciona, mas os comandos UDP não respondem

Pedidos ao OLED agora usam uma fila estática de 24 eventos, sem esperar por mutex da tela. Apenas o loop dos periféricos altera textos, páginas e framebuffer. Se a fila encher, o evento novo é descartado e `esp_oled_events_dropped_total` aumenta, sem prender a tarefa de rede. Um comando de tela pode ter sua atualização visual descartada nessa condição; os demais comandos continuam sendo executados. Esse contador não mede erros de I2C. O timeout I2C é de 25 ms por transação, não um limite total de duração de um quadro inteiro.

Neste projeto a conexão é Wi-Fi. O Prometheus coleta por HTTP/TCP na porta configurada (padrão 9120), enquanto comandos usam UDP 4210. Um scrape atual não confirma que o trajeto UDP entre o computador e

o ESP32 está funcionando. Confirme que `esp_uptime_seconds` aumenta e que `up` está em 1, para não confundir dados antigos do dashboard com coleta atual.

O firmware verifica a abertura do UDP e repete tentativas malsucedidas a cada cinco segundos. Eventos de reconexão, mudança de IP e início/fim do AP solicitam reabertura na tarefa de rede. Erros de socket detectados e falhas no envio também solicitam recuperação. Ausência de pacotes, sozinha, não dispara reaberturas. Os callbacks de Wi-Fi apenas sinalizam a recuperação; não operam o socket.

Após atualizar o firmware, acompanhe estas métricas enquanto envia `alive` para o IP atual da placa, na porta **UDP 4210**:

```
esp_udp_listener_up{port="4210"}
esp_udp_packets_received_total{port="4210"}
esp_udp_replies_sent_total{port="4210"}
esp_udp_send_errors_total{port="4210"}
esp_udp_bind_errors_total{port="4210"}
```

- Pacotes recebidos sem aumento: confirme IP, UDP em vez de TCP, porta 4210, firewall, isolamento de clientes Wi-Fi e o caminho entre computador e placa.
- Pacotes aumentam e respostas também: a placa recebeu e a pilha aceitou a resposta. Confira se o cliente escuta a mesma porta de origem usada no envio.
- Erros de envio ou abertura aumentam: a recuperação local está sendo tentada; consulte também heap, desconexões e contadores de reboot/watchdog.
- `esp_udp_listener_up=1` indica apenas abertura local. Não é um teste de conectividade ponta a ponta nem prova que todo erro silencioso foi detectado.

Todas as métricas UDP têm labels `device` e `port="4210"`. Os contadores ficam na RAM e reiniciam com a placa; `reset_counters` continua limitado aos contadores de boot/watchdog da NVS.

Exemplo no PowerShell:

```
curl.exe http://192.168.1.42:9120/metrics
curl.exe http://192.168.1.42/prometheus.yml
```

Se a conexão for recusada, confira IP, porta salva, firmware gravado e se já passou cerca de um segundo desde o salvamento. Consulte na Serial a linha `[METRICS] Porta ...; interface web na porta 80.`. Confirme que o servidor consegue alcançar a rede do ESP32 e aplique o YAML atualizado.

Consultas úteis:

```
up{job="esp32_wifi"}
esp_heap_free_bytes{job="esp32_wifi"}
rate(application_requests_total{job="esp32_wifi"}[5m])
rate(application_errors_total{job="esp32_wifi"}[5m])
esp_cpu_usage_percent{job="esp32_wifi"}
```

Validação na placa

A compilação não comprova operação em hardware. Após gravar, verifique:

1. Painel na porta 80 e `/metrics` na porta configurada, tanto em STA quanto AP.
2. Salvar porta diferente e manter o painel aberto na porta 80.
3. Trocar a coleta para 80 e depois retornar a outra porta, sem conflito.
4. Reiniciar e confirmar configurações preservadas e incremento de boot.
5. Aplicar o YAML no Prometheus e confirmar `up=1` e as séries disponíveis.
6. Verificar os recursos anteriores: Wi-Fi, UDP, OTA, OLED, botão, LED e NTP.
7. Com Internet e NTP, aguardar a primeira consulta e conferir `esp_weather_up=1`, temperatura, umidade e idade com `source="open_meteo",location="porto_alegre"`.
8. Testar as três consultas do Grafana com `$device=ESP32-WiFi-OLED`. Confirmar os labels `instance`/`job` no Prometheus, e não apenas no texto de `/metrics`.
9. Interromper a Internet e verificar idade crescente, estado de coleta 0 e omissão de temperatura/umidade após uma hora, sem bloquear OLED, UDP ou OTA.
10. Enviar `alive` por UDP 4210 e conferir resposta e aumento de recebimentos/envios. Desconectar/reconectar o Wi-Fi, conferir reabertura e repetir no IP atual. Datagramas acima de 255 bytes devem ser rejeitados sem impedir o próximo comando.
11. Sem tocar no botão, enviar `alive` e `crash_info` durante a rotação das cinco páginas e durante a proteção de tela. Verificar respostas, uptime crescente e `esp_oled_events_dropped_total`. Em novo watchdog sem cortar alimentação, consultar o rastro RTC no próximo boot.
12. Abrir um cliente TCP que não envie o pedido e verificar encerramento após aproximadamente 1,5 s. Testar pedido HTTP fragmentado, formulários de Wi-Fi, Prometheus e terminal, além de scrapes repetidos. Durante os testes, enviar `alive` por UDP e acompanhar uptime, watchdogs e os dois contadores HTTP.
13. Testar um cliente que para de ler a resposta: o envio normal deve ser interrompido pelo prazo de um segundo, sem aumentar/desativar o watchdog.

Referências: modelo de coleta Prometheus, formato de exposição.